

My 10x Solution — Arthur Henrique Lopes Feitosa

Project: Task Manager AI & SRE Triager Platform | **Track:** Backend AI Engineering | **Author:** Arthur Henrique Lopes Feitosa

1. Problem Statement & Solution Overview

Modern engineering and operations teams face high volumes of incoming customer support tickets, operational alerts, and system issues. Triage is often done manually, leading to delayed response times, miscategorization, and lost revenue.

The Solution: An end-to-end, production-grade **AI-Powered Task Management & Autonomous SRE Triage Platform** combining high-throughput REST APIs, relational persistence, secure authentication, background job queues, PDF report generation, and an autonomous LLM triage engine with self-healing validation loops.

2. Core Program Concepts Implemented (8 Concepts)

- 1. High-Performance REST API:** Built with FastAPI in Python 3.11 with automatic OpenAPI/Swagger interactive documentation (/docs) and full task CRUD.
- 2. Relational Database:** Backed by PostgreSQL with indexed primary keys, idempotency constraints, and connection pooling for tasks, triage_jobs, and report_jobs.
- 3. Authentication & Security:** Supabase Auth JWT token issuance, verification, and HTTPBearer dependency injection for protected endpoints.
- 4. Background Job Processing:** Asynchronous execution with FastAPI BackgroundTasks (202 Accepted) and Inngest event-driven DAG workflow orchestration (/api/inngest).
- 5. Automated PDF Report Generation:** On-demand background PDF report generator (/reports/generate) using FPDF/ReportLab with SQL data aggregations and static artifact storage.
- 6. LLM Autonomous Triage & Self-Repair:** LLM SRE Triager Agent classifying tickets into strict taxonomy with Pydantic schemas, 1-shot repair prompts, and quarantine logging.
- 7. Web Scraper Pipeline:** Polite web scraping subsystem (scraper/) with rate limiting and structured dataset exports.
- 8. Interactive Frontend Canvas:** React Flow + TailwindCSS + Vite visual canvas (frontend/) connecting prompt nodes directly to backend execution pipelines.

3. Architecture & Verification

The system follows a clean micro-service and background worker architecture. All endpoints are verified via automated evals (evals/cases.json) and containerized with Docker Compose.

Submitted as the Capstone Project for the FlyRank Backend AI Engineering Track. Repository:
<https://github.com/arthurhenriquelopes/task-manager-api>